VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: Implementation of Memory Allocation Methods

Experiment No: 4 **Date: 27-08-2026**

Aim: Implementation of the following Memory Allocation Methods for fixed partition a) First Fit b) Worst Fit c) Best Fit

A. First Fit

Algorithm:

1. Start.
2. Input the number of memory partitions (m) and their sizes, and the number of processes (n) and their memory requirements.
3. Initialize all processes as unallocated.
4. For each process:
   - Search the partitions from the first partition.
   - If an unallocated partition is large enough, allocate it to the process.
   - Mark the partition as allocated and stop searching.
5. Calculate the Internal Fragmentation for each allocated partition:
   - Fragmentation = Partition Size − Process Size
6. Display the process, allocated partition, partition size, and internal fragmentation.
7. Stop.

B. Worst Fit

Algorithm:

1. Start.
2. Input the number of memory partitions (m) and their sizes, and the number of processes (n) and their memory requirements.
3. Initialize all processes as unallocated.
4. For each process:
   - Search all unallocated partitions.
   - Find the largest partition that is large enough for the process.
   - If a suitable partition is found, allocate it to the process and mark it as allocated.
5. Calculate the Internal Fragmentation:
   - Fragmentation = Partition Size − Process Size
6. Display the process, allocated partition, partition size, and internal fragmentation.
7. Stop.

C. Best Fit

Algorithm:

1. Start.
2. Input the number of memory partitions (m) and their sizes, and the number of processes (n) and their memory requirements.
3. Initialize all processes as unallocated.
4. For each process:
   - Search all unallocated partitions.
   - Find the smallest partition that is large enough for the process.
   - If a suitable partition is found, allocate it to the process and mark it as allocated.
5. Calculate the Internal Fragmentation:
   - Fragmentation = Partition Size − Process Size
6. Display the process, allocated partition, partition size, and internal fragmentation.
7. Stop.

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: _____

_____

Experiment No: ____ **Date:** _____

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: _____

Experiment No: ____ **Date:** _____

A. First Fit

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A62226\n");
   int m, n, bsize[10], psize[10], alloc[10], occupied[10] = {0};
   printf("Enter number of blocks: "); scanf("%d", &m);
   printf("Enter block sizes: ");
   for (int i = 0; i < m; i++) scanf("%d", &bsize[i]);
   printf("Enter number of processes: "); scanf("%d", &n);
   printf("Enter process sizes: ");
   for (int i = 0; i < n; i++) {
    scanf("%d", &psize[i]); alloc[i] = -1;
 }
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
         if (!occupied[j] && bsize[j] >= psize[i]) {
            alloc[i] = j;
            occupied[j] = 1;
            break; }  }    }
   printf("\nProcess\tProcess Size\tBlock No.\tBlock Size\tInternal Frag\n");
   for (int i = 0; i < n; i++) {
      if (alloc[i] != -1)
         printf("P%d\t%d\t\t%d\t\t%d\t\t%d\n", i + 1, psize[i], alloc[i] + 1, bsize[alloc[i]], bsize[alloc[i]] - psize[i]);
      else
         printf("P%d\t%d\t\tNot Allocated\t-\t\t-\n", i + 1, psize[i]);}
   return 0;}
```

Testcases:

```
Roll no: 25071A6226
Enter number of blocks: 5
Enter block sizes: 100 500 200 300 600
Enter number of processes: 4
Enter process sizes: 212 417 112 426

Process Process Size        Block No.         Block Size          Internal Frag
P1      212                 2                 500                 288
P2      417                 5                 600                 183
P3      112                 3                 200                 88
P4      426                 Not Allocated     -                   -
```

```
Roll no: 25071A6226
Enter number of blocks: 4
Enter block sizes: 200 400 600 500
Enter number of processes: 4
Enter process sizes: 150 350 450 100

Process Process Size        Block No.         Block Size          Internal Frag
P1      150                 1                 200                 50
P2      350                 2                 400                 50
P3      450                 3                 600                 150
P4      100                 4                 500                 400
```

```
Roll no: 25071A6226
Enter number of blocks: 5
Enter block sizes: 300 150 200 350 600
Enter number of processes: 4
Enter process sizes: 120 290 180 550

Process Process Size        Block No.         Block Size          Internal Frag
P1      120                 1                 300                 180
P2      290                 4                 350                 60
P3      180                 3                 200                 20
P4      550                 5                 600                 50
```

B. Worst Fit

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A62226\n");
   int m, n, bsize[10], psize[10], alloc[10], occupied[10] = {0};
   printf("Enter number of blocks: "); scanf("%d", &m);
   printf("Enter block sizes: ");
   for (int i = 0; i < m; i++) scanf("%d", &bsize[i]);
   printf("Enter number of processes: "); scanf("%d", &n);
   printf("Enter process sizes: ");
   for (int i = 0; i < n; i++) { scanf("%d", &psize[i]); alloc[i] = -1; }
   for (int i = 0; i < n; i++) {
      int worstIdx = -1;
      for (int j = 0; j < m; j++) {
         if (!occupied[j] && bsize[j] >= psize[i]) {
            if (worstIdx == -1 || bsize[j] > bsize[worstIdx])
               worstIdx = j;}    }
      if (worstIdx != -1) {
         alloc[i] = worstIdx;
         occupied[worstIdx] = 1; }}
   printf("\nProcess\tProcess Size\tBlock No.\tBlock Size\tInternal Frag\n");
   for (int i = 0; i < n; i++) {
      if (alloc[i] != -1)
         printf("P%d\t%d\t\t%d\t\t%d\t\t%d\n", i + 1, psize[i], alloc[i] + 1, bsize[alloc[i]], bsize[alloc[i]] - psize[i]);
      else
         printf("P%d\t%d\t\tNot Allocated\t-\t\t-\n", i + 1, psize[i]); }
   return 0;}
```

Testcases:

```
Roll no: 25071A6226
Enter number of blocks: 5
Enter block sizes: 100 500 200 300 600
Enter number of processes: 4
Enter process sizes: 212 417 112 426

Process  Process Size    Block No.      Block Size     Internal Frag
P1       212             2              500            288
P2       417             5              600            183
P3       112             3              200            88
P4       426             Not Allocated  -              -
```

```
Roll no: 25071A6226
Enter number of blocks: 4
Enter block sizes: 200 400 600 500
Enter number of processes: 4
Enter process sizes: 150 350 450 100

Process  Process Size    Block No.      Block Size     Internal Frag
P1       150             1              200            50
P2       350             2              400            50
P3       450             3              600            150
P4       100             4              500            400
```

```
Roll no: 25071A6226
Enter number of blocks: 5
Enter block sizes: 300 150 200 350 600
Enter number of processes: 4
Enter process sizes: 120 290 180 550

Process  Process Size    Block No.      Block Size     Internal Frag
P1       120             1              300            180
P2       290             4              350            60
P3       180             3              200            20
P4       550             5              600            50
```

C. Best Fit

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A62226\n");
   int m, n, bsize[10], psize[10], alloc[10], occupied[10] = {0};
   printf("Enter number of blocks: "); scanf("%d", &m);
   printf("Enter block sizes: ");
   for (int i = 0; i < m; i++) scanf("%d", &bsize[i]);
   printf("Enter number of processes: "); scanf("%d", &n);
   printf("Enter process sizes: ");
   for (int i = 0; i < n; i++) { scanf("%d", &psize[i]); alloc[i] = -1; }

   for (int i = 0; i < n; i++) {
      int bestIdx = -1;
      for (int j = 0; j < m; j++) {
         if (!occupied[j] && bsize[j] >= psize[i]) {
            if (bestIdx == -1 || bsize[j] < bsize[bestIdx])
               bestIdx = j;
         }
      }
      if (bestIdx != -1) {
         alloc[i] = bestIdx;
         occupied[bestIdx] = 1;
      }
   }
   printf("\nProcess\tProcess Size\tBlock No.\tBlock Size\tInternal Frag\n");
   for (int i = 0; i < n; i++) {
      if (alloc[i] != -1)
         printf("P%d\t%d\t\t%d\t\t%d\t\t%d\n", i + 1, psize[i], alloc[i] + 1, bsize[alloc[i]], bsize[alloc[i]] - psize[i]);
      else
         printf("P%d\t%d\t\tNot Allocated\t-\t\t-\n", i + 1, psize[i]);
   }
   return 0;
}
```

Testcases:

```
Roll no: 25071A6226
Enter number of blocks: 4
Enter block sizes: 200 400 600 500
Enter number of processes: 4
Enter process sizes: 150 350 450 100

Process Process Size    Block No.      Block Size     Internal Frag
P1      150             1              200            50
P2      350             2              400            50
P3      450             3              500            50
P4      100             Not Allocated  -              -
```

```
Roll no: 25071A6226
Enter number of blocks: 5
Enter block sizes: 100 500 200 300 600
Enter number of processes: 4
Enter process sizes: 212 417 112 426

Process Process Size    Block No.      Block Size     Internal Frag
P1      212             4              300            88
P2      417             2              500            83
P3      112             3              200            88
P4      426             5              600            174
```

```
Roll no: 25071A6226
Enter number of blocks: 5
Enter block sizes: 300 150 200 350 600
Enter number of processes: 4
Enter process sizes: 120 290 180 550

Process Process Size    Block No.      Block Size     Internal Frag
P1      120             2              150            30
P2      290             1              300            10
P3      180             3              200            20
P4      550             5              600            50
```

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: Paging Technique Memory Management

Experiment No: 5 **Date:** 27-08-2026

Aim: Implementation of Paging Technique of Memory Management

Algorithm:

1. Start.
2. Input the main memory size (MS) and page size (PS), then calculate the Total Frames.
3. Input the process size and calculate the Total Pages required.
4. If Total Pages > Total Frames, display "Memory insufficient" and stop.
5. Input the frame number for each page and create the Page Table.
6. Input the Logical Address (LA) and calculate:
    - Page Number = LA / PS
    - Offset = LA % PS
7. If the page number is valid:
    - Find the Frame Number from the Page Table.
    - Calculate:
        - Physical Address = (Frame Number × PS) + Offset
    - Display the Page Number, Offset, Frame Number, and Physical Address.
8. Otherwise, display "Invalid Logical Address".
9. Stop

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: _____

Experiment No: ____ **Date:** _____

Program:

```
#include <stdio.h>

int main() {
   printf("Roll no: 25071A62226\n");
   int ms, ps, p_size, total_frames, total_pages;
   int page_table[20];
   int la, p, d, pa;

   printf("Enter main memory size (in bytes): ");
   scanf("%d", &ms);
   printf("Enter page size (in bytes): ");
   scanf("%d", &ps);

   total_frames = ms / ps;
   printf("Total frames available in memory: %d\n\n", total_frames);

   printf("Enter process size (in bytes): ");
   scanf("%d", &p_size);

   total_pages = p_size / ps;
   if (p_size % ps != 0) total_pages++;

   if (total_pages > total_frames) {
      printf("\nMemory Allocation Failed: Not enough frames available.\n");
      return 0;
   }

   printf("Total pages required for process: %d\n", total_pages);
   printf("Enter frame numbers allocated to each page:\n");
   for (int i = 0; i < total_pages; i++) {
      printf("Page %d -> Frame No: ", i);
      scanf("%d", &page_table[i]);
   }

   printf("\nEnter Logical Address to translate: ");
   scanf("%d", &la);

   p = la / ps;
   d = la % ps;

   if (p >= total_pages) {
      printf("\nInvalid Logical Address! (Page number %d exceeds allocated pages)\n", p);
   }
```

```
else {
        pa = (page_table[p] * ps) + d;
        printf("\n--- Address Translation Details ---\n");
        printf("Page Number (p)       : %d\n", p);
        printf("Page Offset (d)       : %d\n", d);
        printf("Allocated Frame No    : %d\n", page_table[p]);
        printf("Physical Address      : %d\n", pa);
    }

    return 0;
}
```

Testcases:

```
Roll no: 25071A6226
Enter main memory size (in bytes): 1024
Enter page size (in bytes): 128
Total frames available in memory: 8

Enter process size (in bytes): 350
Total pages required for process: 3
Enter frame numbers allocated to each page:
Page 0 -> Frame No: 5
Page 1 -> Frame No: 2
Page 2 -> Frame No: 7

Enter Logical Address to translate: 200

--- Address Translation Details ---
Page Number (p)       : 1
Page Offset (d)       : 72
Allocated Frame No    : 2
Physical Address      : 328
```

```
Roll no: 25071A6226
Enter main memory size (in bytes): 2048
Enter page size (in bytes): 256
Total frames available in memory: 8

Enter process size (in bytes): 600
Total pages required for process: 3
Enter frame numbers allocated to each page:
Page 0 -> Frame No: 1
Page 1 -> Frame No: 4
Page 2 -> Frame No: 6

Enter Logical Address to translate: 900

Invalid Logical Address! (Page number 3 exceeds allocated pages)
```

```
Roll no: 25071A6226
Enter main memory size (in bytes): 512
Enter page size (in bytes): 64
Total frames available in memory: 8

Enter process size (in bytes): 700

Memory Allocation Failed: Not enough frames available.
```

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: frame allocation Methods.

Experiment No: 7 **Date: 3-09-2026**

Aim: Simulate frame allocation Methods. A) minimum number of frames B) equal allocation C) proportional allocation

A. minimum number of frames

Algorithm:

1. Start.
2. Input total available frames and minimum frames required per process.
3. Input number of processes and their page requirements.
4. Check whether available frames are sufficient for all processes.
5. Allocate the minimum required frames to each process.
6. Calculate and display the remaining unallocated frames.
7. Stop.

B. equal allocation

Algorithm:

1. Start.
2. Input total number of available frames and number of processes m.
3. Calculate frames allocated to each process:
   Frames per Process = Total Frames / m
4. Calculate remaining frames:
   Leftover = Total Frames % m
5. Display frames allocated to each process and the leftover frames.
6. Stop.

C. proportional allocation

Algorithm:

1. Start.
2. Input total number of frames m and number of processes n.
3. Input page requirement s[i] for each process and calculate total requirement S.
4. Calculate frames allocated to each process:
   Allocation[i] = (s[i] / S) × m
5. Calculate total allocated frames and remaining unallocated frames.
6. Display process requirements, allocated frames, and remaining frames.
7. Stop.

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: _____

Experiment No: ____ **Date:** _____

A. minimum number of frames

Program:

```
#include <stdio.h>

int main() {
    printf("Roll no: 25071A62226\n\n");
    int total_frames, min_per_process, n;
    int page_req[10], allocated[10];

    printf("Enter total frames in physical memory: ");
    scanf("%d", &total_frames);
    printf("Enter minimum frames required per process (by architecture): ");
    scanf("%d", &min_per_process);
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int total_min_needed = n * min_per_process;
    if (total_min_needed > total_frames) {
        printf("\nAllocation Failed! Total available frames (%d) < Required minimum frames (%d).\n",
               total_frames, total_min_needed);
        return 0;
    }

    printf("\nEnter page requirements for each process:\n");
    for (int i = 0; i < n; i++) {
        printf("Process P%d: ", i + 1);
        scanf("%d", &page_req[i]);
        allocated[i] = min_per_process;
    }

    int total_allocated = total_min_needed;
    int remaining = total_frames - total_allocated;

    printf("\n--- Minimum Frame Allocation Result ---\n");
    printf("Process\tPage Req\tAllocated Frames\tStatus\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t\t%s\n",
               i + 1, page_req[i], allocated[i],
               (allocated[i] >= page_req[i]) ? "Satisfied" : "Needs Swap Space");
    }
    printf("\nTotal Frames Allocated: %d / %d", total_allocated, total_frames);
    printf("\nRemaining Unallocated Frames: %d\n", remaining);

    return 0;
}
```

Testcases:

```
Roll no: 25071A6226
Enter total frames in physical memory: 10
Enter minimum frames required per process (by architecture): 2
Enter number of processes: 3
Enter page requirements for each process:
Process P1: 5
Process P2: 2
Process P3: 4

--- Minimum Frame Allocation Result ---
Process Page Req        Allocated Frames        Status
P1      5               2                       Needs Swap Space
P2      2               2                       Satisfied
P3      4               2                       Needs Swap Space

Total Frames Allocated: 6 / 10
Remaining Unallocated Frames: 4
```

```
Roll no: 25071A6226
Enter total frames in physical memory: 5
Enter minimum frames required per process (by architecture): 2
Enter number of processes: 3
Enter page requirements for each process:
Process P1: 10
Process P2: 6
Process P3: 8

--- Minimum Frame Allocation Result ---
Process Page Req        Allocated Frames        Status
P1      10              2                       Needs Swap Space
P2      6               2                       Needs Swap Space
P3      8               2                       Needs Swap Space

Total Frames Allocated: 6 / 5
Remaining Unallocated Frames: -1
```

```
Roll no: 25071A6226
Enter total frames in physical memory: 12
Enter minimum frames required per process (by architecture): 2
Enter number of processes: 3
Enter page requirements for each process:
Process P1: 2
Process P2: 2
Process P3: 2

--- Minimum Frame Allocation Result ---
Process Page Req        Allocated Frames        Status
P1      2               2                       Satisfied
P2      2               2                       Satisfied
P3      2               2                       Satisfied

Total Frames Allocated: 6 / 12
Remaining Unallocated Frames: 6
```

B. equal allocation

Program:

```
#include <stdio.h>

int main() {
   printf("Roll no: 25071A62226\n\n");
   int total_frames, m;
   int page_req[10], allocated[10];

   printf("Enter total frames in physical memory: ");
   scanf("%d", &total_frames);
   printf("Enter number of processes: ");
   scanf("%d", &m);

   if (m <= 0 || total_frames < m) {
      printf("\nAllocation Failed! Insufficient frames for equal allocation.\n");
      return 0;
   }

   printf("Enter page requirements for each process:\n");
   for (int i = 0; i < m; i++) {
      printf("Process P%d: ", i + 1);
      scanf("%d", &page_req[i]);
   }

   int equal_share = total_frames / m;
   int leftover = total_frames % m;

   printf("\n--- Equal Frame Allocation Result ---\n");
   printf("Process\tPage Req\tAllocated Frames\tUnmet Pages\n");
   for (int i = 0; i < m; i++) {
      allocated[i] = equal_share;
      int unmet = (page_req[i] > allocated[i]) ? (page_req[i] - allocated[i]) : 0;
      printf("P%d\t%d\t\t%d\t\t\t%d\n", i + 1, page_req[i], allocated[i], unmet);
   }

   printf("\nTotal Allocated Frames: %d / %d", equal_share * m, total_frames);
   printf("\nLeftover System Frames: %d\n", leftover);

   return 0;
}
```

Testcases:

```
Roll no: 25071A6226

Enter total frames in physical memory: 63
Enter number of processes: 3
Enter page requirements for each process:
Process P1: 10
Process P2: 127
Process P3: 18

--- Equal Frame Allocation Result ---
Process Page Req        Allocated Frames        Unmet Pages
P1      10              21                      0
P2      127             21                      106
P3      18              21                      0

Total Allocated Frames: 63 / 63
Leftover System Frames: 0
```

```
Roll no: 25071A6226

Enter total frames in physical memory: 100
Enter number of processes: 3
Enter page requirements for each process:
Process P1: 40
Process P2: 25
Process P3: 60

--- Equal Frame Allocation Result ---
Process Page Req        Allocated Frames        Unmet Pages
P1      40              33                      7
P2      25              33                      0
P3      60              33                      27

Total Allocated Frames: 99 / 100
Leftover System Frames: 1
```

```
Roll no: 25071A6226

Enter total frames in physical memory: 60
Enter number of processes: 3
Enter page requirements for each process:
Process P1: 15
Process P2: 12
Process P3: 18

--- Equal Frame Allocation Result ---
Process Page Req        Allocated Frames        Unmet Pages
P1      15              20                      0
P2      12              20                      0
P3      18              20                      0

Total Allocated Frames: 60 / 60
Leftover System Frames: 0
```

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: _____

Experiment No: ____ **Date:** _____

C. proportional allocation

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A62226\n\n");
   int m, n;
   int s[10], a[10];
   int S = 0, total_allocated = 0;

   printf("Enter total frames available in physical memory: ");
   scanf("%d", &m);
   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter size (in pages/virtual memory) for each process:\n");
   for (int i = 0; i < n; i++) {
      printf("Process P%d size: ", i + 1);
      scanf("%d", &s[i]);
      S += s[i];
   }

   if (S == 0) {
      printf("\nInvalid total process size.\n");
      return 0;
   }
   printf("\n--- Proportional Frame Allocation Result ---\n");
   printf("Total Virtual Memory Required (S) = %d pages\n\n", S);
   printf("Process\tSize (s_i)\tAllocated Frames (a_i)\tPercentage\n");

   for (int i = 0; i < n; i++) {
      a[i] = (int)(((double)s[i] / S) * m);
      total_allocated += a[i];
      double pct = ((double)s[i] / S) * 100.0;
      printf("P%d\t%d\t\t%d\t\t\t%.2f%%\n", i + 1, s[i], a[i], pct);
   }
   printf("\nTotal Frames Allocated: %d / %d", total_allocated, m);
   printf("\nUnallocated Buffer Frames: %d\n", m - total_allocated);

   return 0;
}
```

Testcases:

```
Roll no: 25071A6226

Enter total frames available in physical memory: 62
Enter number of processes: 2
Enter size (in pages/virtual memory) for each process:
Process P1 size: 10
Process P2 size: 127

--- Proportional Frame Allocation Result ---
Total Virtual Memory Required (S) = 137 pages

Process Size (s_i)      Allocated Frames (a_i)  Percentage
P1      10              4                       7.30%
P2      127             57                      92.70%

Total Frames Allocated: 61 / 62
Unallocated Buffer Frames: 1
```

```
Roll no: 25071A6226

Enter total frames available in physical memory: 100
Enter number of processes: 3
Enter size (in pages/virtual memory) for each process:
Process P1 size: 50
Process P2 size: 50
Process P3 size: 50

--- Proportional Frame Allocation Result ---
Total Virtual Memory Required (S) = 150 pages

Process Size (s_i)      Allocated Frames (a_i)  Percentage
P1      50              33                      33.33%
P2      50              33                      33.33%
P3      50              33                      33.33%

Total Frames Allocated: 99 / 100
Unallocated Buffer Frames: 1
```

```
Roll no: 25071A6226

Enter total frames available in physical memory: 60
Enter number of processes: 3
Enter size (in pages/virtual memory) for each process:
Process P1 size: 20
Process P2 size: 50
Process P3 size: 50

--- Proportional Frame Allocation Result ---
Total Virtual Memory Required (S) = 100 pages

Process Size (s_i)      Allocated Frames (a_i)  Percentage
P1      20              12                      20.00%
P2      30              18                      30.00%
P3      50              30                      50.00%

Total Frames Allocated: 60 / 60
Unallocated Buffer Frames: 0
```

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: Disk Scheduling Algorithms a) FCFS b) SSTF c) SCAN

Experiment No: 8 **Date:** 10-09-2026

Aim: Simulate the following Disk scheduling algorithms. A) FCFS B) SSTF C) SCAN

A) FCFS

Algorithm:

1. Start.
2. Input the number of disk requests (n), request sequence, and initial head position.
3. Initialize Total Head Movement (THM) = 0.
4. For each request:
   - Calculate the distance between the current head and the request.
   - Add the distance to THM.
   - Move the head to the requested position.
5. Calculate:
   - Average Seek Time = THM / n
6. Display the Total Head Movement and Average Seek Time.
7. Stop.

B) SSTF

Algorithm:

1. **Start**
2. Input number of requests n, request sequence, and initial head position.
3. Initialize THM = 0 and mark all requests as unvisited.
4. Repeat n times:
   - Find the unvisited request closest to the current head.
   - Mark it as visited.
   - Add the distance to THM.
   - Move the head to that request.
5. Calculate:
   - Average Seek Time = THM / n
6. Display Total Head Movement and Average Seek Time.
7. **Stop**.

c) SCAN

Algorithm:

1. **Start**.
2. Input number of requests n, request sequence, initial head position, disk size, and direction (LEFT or RIGHT).
3. Sort the request sequence in ascending order.
4. Initialize THM = 0 and divide requests based on the current head position.
5. If direction is RIGHT:
6. Service requests to the right in ascending order.
7. Move the head to disk_size - 1.
8. Reverse direction and service remaining requests in descending order.
9. If direction is LEFT:
10. Service requests to the left in descending order.
11. Move the head to 0.
12. Reverse direction and service remaining requests in ascending order.
13. Calculate Average Seek Time = THM / n and display Total Head Movement and Average Seek Time.
14. **Stop.**

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: _____

_____

Experiment No: ____ **Date:** _____

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

A) FCFS

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
   printf("Roll no: 25071A62226\n");
   int n, head, req[20], thm = 0;
   printf("Enter number of requests: "); scanf("%d", &n);
   printf("Enter request sequence: ");
   for (int i = 0; i < n; i++) scanf("%d", &req[i]);
   printf("Enter initial head position: "); scanf("%d", &head);

   printf("\nSeek Sequence: %d", head);
   for (int i = 0; i < n; i++) {
      thm += abs(req[i] - head);
      head = req[i];
      printf(" -> %d", head);
   }
   printf("\nTotal Head Movement = %d\nAverage Seek Time = %.2f\n", thm, (float)thm / n);
   return 0;
}
```

Testcases:

```
Roll no: 25071A6226
Enter number of requests: 8
Enter request sequence: 98 183 37 122 14 124 65 67
Enter initial head position: 53

Seek Sequence: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total Head Movement = 640
Average Seek Time = 80.00
```

```
Roll no: 25071A6226
Enter number of requests: 5
Enter request sequence: 176 79 34 60 92
Enter initial head position: 50

Seek Sequence: 50 -> 176 -> 79 -> 34 -> 60 -> 92
Total Head Movement = 328
Average Seek Time = 65.60
```

```
Roll no: 25071A6226
Enter number of requests: 6
Enter request sequence: 10 150 20 140 30 130
Enter initial head position: 100

Seek Sequence: 100 -> 10 -> 150 -> 20 -> 140 -> 30 -> 130
Total Head Movement = 710
Average Seek Time = 118.33
```

Name of the Experiment:

Experiment No: **Date:**

25071A62226

B) SSTF

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
   printf("Roll no: 25071A62226\n");
   int n, head, req[20], visited[20] = {0}, thm = 0;
   printf("Enter number of requests: "); scanf("%d", &n);
   printf("Enter request sequence: ");
   for (int i = 0; i < n; i++) scanf("%d", &req[i]);
   printf("Enter initial head position: "); scanf("%d", &head);

   printf("\nSeek Sequence: %d", head);
   for (int count = 0; count < n; count++) {
      int min_dist = 1e9, idx = -1;
      for (int i = 0; i < n; i++) {
         if (!visited[i] && abs(req[i] - head) < min_dist) {
            min_dist = abs(req[i] - head);
            idx = i;
         }
      }
      visited[idx] = 1;
      thm += min_dist;
      head = req[idx];
      printf(" -> %d", head);
   }
   printf("\nTotal Head Movement = %d\nAverage Seek Time = %.2f\n", thm, (float)thm / n);
   return 0;
}
```

Testcases:

Testcases:

```
Roll no: 25071A6226
Enter number of requests: 8
Enter request sequence: 98 183 37 122 14 124 65 67
Enter initial head position: 53

Seek Sequence: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total Head Movement = 640
Average Seek Time = 80.00
```

```
Roll no: 25071A6226
Enter number of requests: 5
Enter request sequence: 176 79 34 60 92
Enter initial head position: 50

Seek Sequence: 50 -> 176 -> 79 -> 34 -> 60 -> 92
Total Head Movement = 328
Average Seek Time = 65.60
```

```
Roll no: 25071A6226
Enter number of requests: 6
Enter request sequence: 10 150 20 140 30 130
Enter initial head position: 100

Seek Sequence: 100 -> 10 -> 150 -> 20 -> 140 -> 30 -> 130
Total Head Movement = 710
Average Seek Time = 118.33
```

C) SCAN

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
   printf("Roll no: 25071A62226\n");
   int n, h, r[30], s, d, thm = 0, p = 0;
   printf("Enter number of requests: "); scanf("%d", &n);
   printf("Enter request sequence: "); for (int i = 0; i < n; i++) scanf("%d", &r[i]);
   printf("Enter initial head position: "); scanf("%d", &h);
   printf("Enter total disk size: "); scanf("%d", &s);
   printf("Enter direction (1 for High/Right, 0 for Low/Left): "); scanf("%d", &d);
   r[n++] = h; r[n++] = d ? s - 1 : 0;
   for (int i = 0; i < n - 1; i++)
      for (int j = 0; j < n - i - 1; j++)
         if (r[j] > r[j + 1]) { int t = r[j]; r[j] = r[j + 1]; r[j + 1] = t; }
   while (r[p] != h) p++;
   printf("\nSeek Sequence: %d", h);
   int cur = h, step = d ? 1 : -1, start = d ? p + 1 : p - 1, end = d ? n : -1;
   for (int pass = 0; pass < 2; pass++) {
      for (int i = start; i != end; i += step) { thm += abs(r[i] - cur); cur = r[i]; printf(" -> %d", cur); }
      step = -step; start = (d == (pass == 0)) ? p - 1 : p + 1; end = (d == (pass == 0)) ? -1 : n;
   }
   printf("\nTotal Head Movement = %d\nAverage Seek Time = %.2f\n", thm, (float)thm / (n - 2));
   return 0;
}
```

Testcases:

```
Roll no: 25071A6226
Enter number of requests: 8
Enter request sequence: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter total disk size: 200
Enter direction (1 for High/Right, 0 for Low/Left): 1

Seek Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 37 -> 14
Total Head Movement = 331
Average Seek Time = 41.38
```

```
Roll no: 25071A6226
Enter number of requests: 5
Enter request sequence: 176 79 34 60 92
Enter initial head position: 50
Enter total disk size: 200
Enter direction (1 for High/Right, 0 for Low/Left): 1

Seek Sequence: 50 -> 60 -> 79 -> 92 -> 176 -> 199 -> 34
Total Head Movement = 314
Average Seek Time = 62.80
```

```
Roll no: 25071A6226
Enter number of requests: 6
Enter request sequence: 10 150 20 140 30 130
Enter initial head position: 100
Enter total disk size: 200
Enter direction (1 for High/Right, 0 for Low/Left): 1

Seek Sequence: 100 -> 130 -> 140 -> 150 -> 199 -> 30 -> 20 -> 10
Total Head Movement = 288
Average Seek Time = 48.00
```

Aim: Simulate the following Disk scheduling algorithms. A) C-SCAN B) LOOK C) C-LOOK

A) C- SCAN

Algorithm:

1. Start.
2. Input number of requests n, request sequence, initial head position, and disk size.
3. Sort the request sequence in ascending order.
4. If direction is RIGHT:
    - Service requests from the current head towards disk_size - 1.
    - Move the head directly to 0.
    - Continue servicing the remaining requests in ascending order.
5. Calculate Total Head Movement and Average Seek Time.
6. Display the results.
7. Stop.

B) LOOK

Algorithm:

1. **Start.**
2. Input number of requests n, request sequence, initial head position, and direction.
3. Sort the request sequence in ascending order.
4. If direction is RIGHT:
    - Service requests to the right up to the maximum requested track.
    - Reverse direction and service the remaining requests to the left.
5. If direction is LEFT:
    - Service requests to the left up to the minimum requested track.
    - Reverse direction and service the remaining requests to the right.
6. Calculate Total Head Movement and Average Seek Time.
7. Display the results.
8. **Stop.**

B) C-LOOK

Algorithm:

1. **Start**.
2. Input number of requests n, request sequence, initial head position, and direction.
3. Sort the request sequence in ascending order.
4. If direction is RIGHT:
    - Service requests to the right up to the maximum requested track.
    - Jump directly to the minimum requested track.
    - Service the remaining requests up to the track before the starting head.
5. Calculate Total Head Movement and Average Seek Time.
6. Display the results.
7. **Stop.**

C) C-SCAN

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
   printf("Roll no: 25071A62226\n");
   int n, head, req[30], disk_size, thm = 0;
   printf("Enter number of requests: "); scanf("%d", &n);
   printf("Enter request sequence: ");
   for (int i = 0; i < n; i++) scanf("%d", &req[i]);
   printf("Enter initial head position: "); scanf("%d", &head);
   printf("Enter total disk size: "); scanf("%d", &disk_size);

   req[n++] = head;
   req[n++] = 0;
   req[n++] = disk_size - 1;

   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (req[j] > req[j + 1]) {
            int t = req[j]; req[j] = req[j + 1]; req[j + 1] = t;}}
    }
   int pos = 0;
   for (int i = 0; i < n; i++) if (req[i] == head) { pos = i; break; }

   printf("\nSeek Sequence: %d", head);
   int cur = head;
   for (int i = pos + 1; i < n; i++) {
      thm += abs(req[i] - cur); cur = req[i]; printf(" -> %d", cur);
  }
   for (int i = 0; i < pos; i++) {
      thm += abs(req[i] - cur); cur = req[i]; printf(" -> %d", cur);}
   printf("\nTotal Head Movement = %d\nAverage Seek Time = %.2f\n", thm, (float)thm / (n - 3));
   return 0;
}
```

Testcases:

```
Roll no: 25071A6226
Enter number of requests: 5
Enter request sequence: 176 79 34 60 92
Enter initial head position: 50
Enter total disk size: 200

Seek Sequence: 50 -> 60 -> 79 -> 92 -> 176 -> 199 -> 0 -> 34
Total Head Movement = 382
Average Seek Time = 76.40
```

```
Roll no: 25071A6226
Enter number of requests: 6
Enter request sequence: 10 150 20 140 30 130
Enter initial head position: 100
Enter total disk size: 200

Seek Sequence: 100 -> 130 -> 140 -> 150 -> 199 -> 0 -> 10 -> 20 -> 30
Total Head Movement = 378
Average Seek Time = 63.00
```

```
Roll no: 25071A6226
Enter number of requests: 8
Enter request sequence: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter total disk size: 200

Seek Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 0 -> 14 -> 37
Total Head Movement = 382
Average Seek Time = 47.75
```

C) LOOK

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
   printf("Roll no: 25071A62226\n");
   int n, head, req[25], thm = 0;
   printf("Enter number of requests: "); scanf("%d", &n);
   printf("Enter request sequence: ");
   for (int i = 0; i < n; i++) scanf("%d", &req[i]);
   printf("Enter initial head position: "); scanf("%d", &head);

   req[n++] = head;
   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (req[j] > req[j + 1]) {
            int t = req[j]; req[j] = req[j + 1]; req[j + 1] = t;
         }
      }
   }

   int pos = 0;
   for (int i = 0; i < n; i++) if (req[i] == head) { pos = i; break; }

   printf("\nSeek Sequence: %d", head);
   int cur = head;
   for (int i = pos + 1; i < n; i++) {
      thm += abs(req[i] - cur); cur = req[i]; printf(" -> %d", cur);
   }
   for (int i = pos - 1; i >= 0; i--) {
      thm += abs(req[i] - cur); cur = req[i]; printf(" -> %d", cur);
   }

   printf("\nTotal Head Movement = %d\nAverage Seek Time = %.2f\n", thm, (float)thm / (n - 1));
   return 0;
}
```

Testcases:

```
Roll no: 25071A6226
Enter number of requests: 6
Enter request sequence: 10 150 20 140 30 130
Enter initial head position: 100

Seek Sequence: 100 -> 130 -> 140 -> 150 -> 30 -> 20 -> 10
Total Head Movement = 190
Average Seek Time = 31.67
```

```
Roll no: 25071A6226
Enter number of requests: 5
Enter request sequence: 176 79 34 60 92
Enter initial head position: 50

Seek Sequence: 50 -> 60 -> 79 -> 92 -> 176 -> 34
Total Head Movement = 268
Average Seek Time = 53.60
```

```
Roll no: 25071A6226
Enter number of requests: 8
Enter request sequence: 98 183 37 122 14 124 65 67
Enter initial head position: 53

Seek Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 37 -> 14
Total Head Movement = 299
Average Seek Time = 37.38
```

C) C-LOOK

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
   printf("Roll no: 25071A62226\n");
   int n, head, req[25], thm = 0;
   printf("Enter number of requests: "); scanf("%d", &n);
   printf("Enter request sequence: ");
   for (int i = 0; i < n; i++) scanf("%d", &req[i]);
   printf("Enter initial head position: "); scanf("%d", &head);

   req[n++] = head;
   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (req[j] > req[j + 1]) {
            int t = req[j]; req[j] = req[j + 1]; req[j + 1] = t;
         }
      }
   }

   int pos = 0;
   for (int i = 0; i < n; i++) if (req[i] == head) { pos = i; break; }

   printf("\nSeek Sequence: %d", head);
   int cur = head;
   for (int i = pos + 1; i < n; i++) {
      thm += abs(req[i] - cur); cur = req[i]; printf(" -> %d", cur);
   }
   for (int i = 0; i < pos; i++) {
      thm += abs(req[i] - cur); cur = req[i]; printf(" -> %d", cur);
   }

   printf("\nTotal Head Movement = %d\nAverage Seek Time = %.2f\n", thm, (float)thm / (n - 1));
   return 0;
}
```

Testcases:

```
Roll no: 25071A6226
Enter number of requests: 8
Enter request sequence: 98 183 37 122 14 124 65 67
Enter initial head position: 53

Seek Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 14 -> 37
Total Head Movement = 322
Average Seek Time = 40.25
```

```
Roll no: 25071A6226
Enter number of requests: 5
Enter request sequence: 176 79 34 60 92
Enter initial head position: 50

Seek Sequence: 50 -> 60 -> 79 -> 92 -> 176 -> 34
Total Head Movement = 268
Average Seek Time = 53.60
```

```
Roll no: 25071A6226
Enter number of requests: 6
Enter request sequence: 10 150 20 140 30 130
Enter initial head position: 100

Seek Sequence: 100 -> 130 -> 140 -> 150 -> 10 -> 20 -> 30
Total Head Movement = 210
Average Seek Time = 35.00
```